

nota.ask

Interface Inteligente para Consulta de Arquivos CSV

Relatório Técnico — Desafio 4

Agente inteligente que responde perguntas em linguagem natural sobre notas fiscais armazenadas em CSV. A pergunta do usuário é traduzida em SQL, o SQL é executado sobre os dados e a resposta é montada a partir do resultado real da consulta — nunca inventada pelo modelo.

Campo	Valor
Programa	InsurMinds — Instituto de IA Aplicada (I2A2)
Desafio	Desafio 4 — Interface Inteligente para Consulta de CSV
Framework de agentes	LangChain
Stack	Python, Streamlit, DuckDB, LangChain, LLM (Ollama / API compatível)
Licença	MIT

Grupo: <preencher nome do grupo> | Integrantes: <preencher>

1. Objetivo e visão geral

O objetivo é permitir que qualquer pessoa, sem conhecer SQL, consulte grandes volumes de notas fiscais em CSV usando apenas linguagem natural. A solução recebe um arquivo ZIP com os CSVs e um dicionário de dados, carrega tudo automaticamente e disponibiliza uma interface de perguntas. Um agente construído com LangChain interpreta a pergunta, gera a consulta SQL correspondente, executa sobre os dados e apresenta a resposta em texto, tabela e gráfico.

A decisão de projeto central é **text-to-SQL sobre DuckDB**: o modelo de linguagem escreve a consulta, mas quem calcula os números é o banco. Isso garante respostas fiéis aos dados e atende à regra do desafio de que as respostas sejam produzidas pela aplicação, e não digitadas manualmente em uma LLM.

2. Arquitetura da solução

2.1 Componentes

- **app.py** (Streamlit) — as duas interfaces exigidas: aba **Carga** (Interface A) e aba **Consulta** (Interface B).
- **ingestion.py** — descompacta o ZIP, identifica CSVs e dicionário, normaliza os nomes de coluna, ajusta tipos (valor, quantidade e data) e carrega cada CSV como tabela em um arquivo DuckDB.
- **agent.py** — o agente: gera o SQL via LangChain, valida, corrige e executa a consulta, e resume o resultado em português.
- **DuckDB** — motor analítico local que executa o SQL sobre as tabelas.

2.2 Fluxo de funcionamento

```
ZIP → descompacta → DuckDB (schema + dicionário) → pergunta em linguagem natural →  
geração de SQL (LangChain) → validação e correção → execução no DuckDB → resposta  
(texto + tabela + gráfico).
```

Após o processamento do ZIP, a interface de consulta é habilitada automaticamente, conforme pedido no enunciado.

3. Descrição do agente

O agente é implementado com LangChain (modelos de chat, *ChatPromptTemplate* e *StrOutputParser*). Ele opera em três passos e é resiliente a erros:

3.1 Geração do SQL

Recebe a pergunta, o schema real das tabelas e o dicionário de dados, e produz uma única consulta SELECT. O prompt instrui o modelo a usar apenas nomes existentes, ordenar corretamente perguntas de ranking, agrupar corretamente e tratar datas com *strftime*.

3.2 Validação e correção automática

Antes e depois da execução, o SQL passa por camadas determinísticas que não dependem do modelo:

- **Guard de leitura**: bloqueia qualquer comando que não seja SELECT/WITH (INSERT, UPDATE, DELETE, DROP, etc.).

- **Corretor de nomes:** aproxima nomes de coluna quase corretos aos nomes reais do schema (por similaridade).
- **Correção por tabela:** conserta coluna pedida no alias/tabela errado em JOINS.
- **Correção pelo erro do banco:** lê a sugestão do DuckDB ("Candidate bindings") e aplica o nome correto.
- **Correção de tipo:** ajusta agregações quando o tipo da coluna exige.
- **Retentativa:** em caso de falha, devolve o erro ao modelo para nova tentativa (até 5 vezes).

3.3 Resposta

O resultado da consulta é resumido em português de forma fiel — o modelo é instruído a não reordenar nem inventar valores. A resposta final combina texto, a tabela de dados e, quando aplicável, um gráfico de barras.

4. Preparação automática dos dados

A ingestão trata as particularidades de CSV de nota fiscal brasileira, o que reduz muito os erros na etapa de consulta:

- Detecção de codificação (UTF-8, depois latin-1) e do separador de colunas.
- Normalização de nomes de coluna, com transliteração de acentos (ex.: DESTINATÁRIO → DESTINATARIO).
- Colunas de identificador (CNPJ, CPF, chave, código) carregadas como texto, preservando zeros à esquerda e evitando erros de junção.
- Colunas de valor e quantidade convertidas para número, tratando o formato brasileiro (1.234,56 → 1234.56).
- Colunas de data convertidas para o tipo data, habilitando agrupamento por mês/ano.

5. Perguntas realizadas e respostas

Execuções reais sobre a base **202401_NFs** (100 notas no cabeçalho, 565 itens). Para cada pergunta constam o SQL gerado e executado pela aplicação e a resposta produzida a partir do resultado.

Pergunta 1 — Quantas notas fiscais existem no total?

```
SELECT COUNT(*) AS total_notas FROM "t_202401_nfs_cabecalho"
```

Resposta: Existem **100** notas fiscais no total.

Pergunta 2 — Quais os 5 fornecedores com maior valor total de notas?

```
SELECT "RAZAO_SOCIAL_EMITENTE", SUM("VALOR_NOTA_FISCAL") AS total  
FROM "t_202401_nfs_cabecalho"  
GROUP BY "RAZAO_SOCIAL_EMITENTE" ORDER BY total DESC LIMIT 5
```

Fornecedor	Valor total (R\$)
CHEMYUNION LTDA	1.292.418,75
LABORATORIOS B.BRAUN S.A	726.081,60
XCMG BRASIL INDUSTRIA LTDA	330.000,00
EDITORA FTD S.A.	292.486,11
MALTACARE DISTRIBUIDORA LTDA	122.202,60

Resposta: O maior fornecedor por valor é a CHEMYUNION LTDA (R\$ 1.292.418,75), seguida por LABORATORIOS B.BRAUN S.A e XCMG BRASIL.

Pergunta 3 — Qual o valor total das notas por mês?

```
SELECT strftime("DATA_EMISSAO", '%Y-%m') AS mes, SUM("VALOR_NOTA_FISCAL") AS total  
FROM "t_202401_nfs_cabecalho" GROUP BY mes ORDER BY mes
```

Mês	Valor total (R\$)
2024-01	3.188.716,98
2024-02	1.842,00
2024-03	27.451,85
2024-04	2.784,05
2024-05	2.103,20
2024-08	7.060,83
2024-09	18.055,38
2024-10	23.388,03
2024-11	56.126,88
2024-12	44.225,64

Resposta: A maior concentração de valor está em janeiro de 2024 (R\$ 3.188.716,98); os demais meses têm volumes bem menores.

Pergunta 4 — Quais os 5 produtos com maior quantidade comprada?

```
SELECT "DESCRICAO_DO_PRODUTO_SERVICO", SUM("QUANTIDADE") AS total_qtd  
FROM "t_202401_nfs_itens"  
GROUP BY "DESCRICAO_DO_PRODUTO_SERVICO" ORDER BY total_qtd DESC LIMIT 5
```

Produto	Qtd.
DIPIFARMA INJETAVEL (DIPIRONA 500MG/ML) 2ML	51.000
HGC 4 Branco/Branco-300-CP	33.000
SAPATILHA DESCARTAVEL - COR BRANCA - NAO ESTERIL	23.000
SINVASTATINA 40 MG CP HOSP (G)	21.600
TERIFLUNOMIDA 14 MG COM REV CT BL AL X 30 (G)	11.790

Resposta: O produto de maior quantidade é a Dipirona injetável (51.000 unidades), seguida do item HGC 4 Branco (33.000).

Pergunta 5 (bônus) — Quais os 5 estados que mais emitiram por valor?

```
SELECT "UF_EMITENTE", SUM("VALOR_NOTA_FISCAL") AS total
FROM "t_202401_nfs_cabecalho" GROUP BY "UF_EMITENTE" ORDER BY total DESC LIMIT 5
```

UF	Valor total (R\$)
SP	1.733.572,82
RJ	828.244,22
MG	364.802,36
MS	148.439,10
RS	103.115,48

Resposta: São Paulo lidera em valor emitido (R\$ 1.733.572,82), seguido do Rio de Janeiro e de Minas Gerais.

6. Tecnologias utilizadas

Componente	Tecnologia	Papel
Interface	Streamlit	Carga do ZIP e tela de perguntas/respostas
Framework de agentes	LangChain	Geração do SQL e resumo da resposta
Modelo de linguagem	Ollama (local) ou API compatível	Traduz a pergunta em SQL
Banco analítico	DuckDB	Executa o SQL sobre os CSVs
Processamento	pandas	Leitura e ajuste de tipos dos CSVs

O provedor de modelo é configurável por variável de ambiente, sem alterar código: Ollama local (execução 100% offline), ou um endpoint compatível com a API da OpenAI. Chaves e credenciais ficam em arquivo `.env`, fora do código e do repositório.

7. Conformidade com os requisitos

Requisito do desafio	Como é atendido
Upload de ZIP com CSVs	Aba Carga (Interface A)
Processamento automático	Ingestão carrega os CSVs no DuckDB ao processar
Interface de perguntas em linguagem natural	Aba Consulta (Interface B)
Pelo menos um agente inteligente	Agente LangChain gera e executa o SQL
Respostas corretas a partir dos dados	SQL executado sobre os dados reais
Respostas em texto, tabela ou gráfico	As três formas na aba Consulta
Framework obrigatório	LangChain
Respostas produzidas pela aplicação	A LLM só gera o SQL; o DuckDB calcula

8. Limitações e evolução

- A qualidade das respostas depende do modelo de linguagem; modelos muito pequenos podem errar consultas complexas (JOIN, agrupamento). As camadas de correção determinística reduzem, mas não eliminam, esses casos.
- A conversão de tipos é guiada pelo nome das colunas; bases com nomenclatura muito diferente podem exigir ajuste.
- Evolução natural: memória de conversas, cache de consultas frequentes e um agente revisor dedicado para validar o SQL antes da execução.

Projeto licenciado sob a licença MIT. Código-fonte e instruções de instalação e execução no arquivo README.md do repositório.